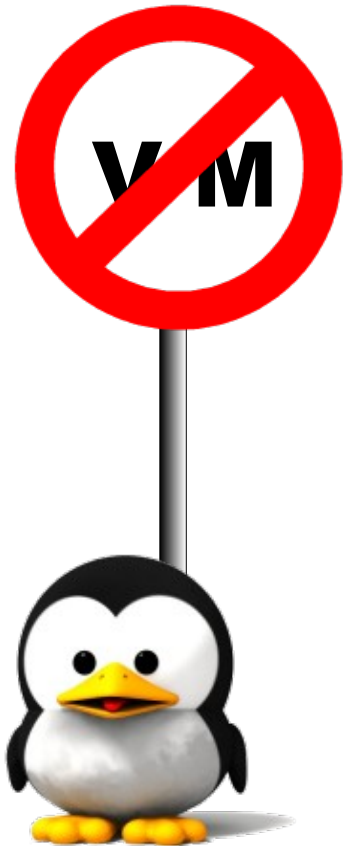


Introduction to uClinux



Introduction to uClinux

Michael Opdenacker

Free Electrons

<http://free-electrons.com>

Created with [OpenOffice.org](http://openoffice.org) 2.x

Thanks to Nicolas Rougier (Copyright 2003, <http://webloria.loria.fr/~rougier/>) for the Tux image



Free Electrons

Introduction to uClinux

© Copyright 2004-2008, Free Electrons

Creative Commons Attribution-ShareAlike 2.5 license

<http://free-electrons.com>



Jul 25, 2025

Rights to copy



Attribution – ShareAlike 2.5

You are free

- to copy, distribute, display, and perform the work
- to make derivative works
- to make commercial use of the work

Under the following conditions

Attribution. You must give the original author credit.

BY: Share Alike. If you alter, transform, or build upon this work you may distribute the resulting work only under a license identical to this one.

CC For any reuse or distribution, you must make clear to others the license terms of this work.

- Any of these conditions can be waived if you get permission from the copyright holder.

Your fair use and other rights are in no way affected by the above.

License text: <http://creativecommons.org/licenses/by-sa/2.5/legalcode>

© Copyright 2004-2008

Free Electrons

feedback@free-electrons.com

Document sources, updates and translations:

<http://free-electrons.com/articles/uclinux>

Corrections, suggestions, contributions and translations are welcome!



Best viewed with...

This document is best viewed with a recent PDF reader or with OpenOffice.org itself!

- ▶ Take advantage of internal or external hyperlinks. So, don't hesitate to click on them!
- ▶ Find pages quickly thanks to automatic search.
- ▶ Use thumbnails to navigate in the document in a quick way.

If you're reading a paper or HTML copy, you should get your copy in PDF or OpenOffice.org format on <http://free-electrons.com/articles/uclinux!>



Contents

- ▶ uClinux project overview
- ▶ uClinux implementation details
- ▶ Using uClinux



Acronyms

- ▶ MMU: Memory Management Unit
- ▶ MPU: Memory Protection Unit
Only implements memory protection, not virtual memory.
- ▶ GOT: Global Offset Table - Used in executable formats.
- ▶ ELF: Executable and Linkable Format
A file format describing executables, object code, shared libraries and core dumps. The OS uses it to know how to load executables and shared libraries.
- ▶ PIC: Position Independent Code
Object code without absolute addresses, which can execute at different locations in memory.
See http://en.wikipedia.org/wiki/Position_independent_code



The MMU job

MMUs included in many general purpose processors available today

- ▶ Virtual to physical address translation.

Allows processes to run in their own virtual contiguous address space. No need for relocating process addresses. Possible to expand the address space of a running process.

The MMU raises an exception when no physical address is available, making it possible to implement swapping to disk.

- ▶ Address protection

Actually done by the MPU available in most MMUs.

Prevents processes from accessing unauthorized memory addresses.

This is also used to implement demand paging (using page faults to load in RAM only the addresses actually accessed) and swapping.

Note that some systems just have an MPU, but no MMU.



Introduction to uClinux

uClinux project overview



Free Electrons

Introduction to uClinux
© Copyright 2004-2008, Free Electrons
Creative Commons Attribution-ShareAlike 2.5 license
<http://free-electrons.com>

Jul 25, 2025



The uClinux project

<http://www.uclinux.org/> - Linux for micro-controllers

Deliveries:

- ▶ **Linux** kernel supporting MMU-less processors (at least 32 bit) Supported in mainstream sources or through patches.
- ▶ Software distribution (source only):
<http://www.uclinux.org/pub/uClinux/dist/>
- ▶ **uClibc**: a lightweight though highly compatible C library. Now an independent project. Used by **Linux** too!
- ▶ Cross-compiling toolchains.



uClinux devices

Just a few examples!

Send us more!
uClinux is often so deeply embedded
that it's difficult to identify



Multimedia



Apple iPod (not shipped with uClinux)



Sigma Designs EM8500 based DVD players

Tiny Single Board Computers



C Data Solutions
CF computer



Simtec SBCs

Network devices



SnapGear LITE2 VPN/Router



StarDot NetCam



picotux RJ45 size
computer



Aplio/PRO IP Phone

Industrial



IntelliCom remote control system



uClinux history

- ▶ First release in 1998 ([Linux 2.0](#)), for the Motorola 68000 processor. Demonstrated on Palm Pilot III.
- ▶ 1999: Motorola ColdFire support
- ▶ 2001: Linux 2.4 support. ARM7 support
- ▶ 2004: Linux 2.6 support for ARM
- ▶ 2008: You're reading this document



Reasons for using uClinux (1)

- ▶ **Linux**
Built-in IP connectivity, reliability, portability, filesystems, free software...
- ▶ **Lightweight**
Full **Linux** 2.6 kernel under 300K, binaries much smaller with **uClibc**.
- ▶ **XIP (Execute In Place)**
Don't have to load executables in RAM. May run slower though.
- ▶ **Cheaper**
MMU-less arm cores are smaller.
- ▶ **Sufficient**
A large number of embedded systems applications can do without an MMU.
- ▶ **Faster**
Faster context switches: no cache flushes.



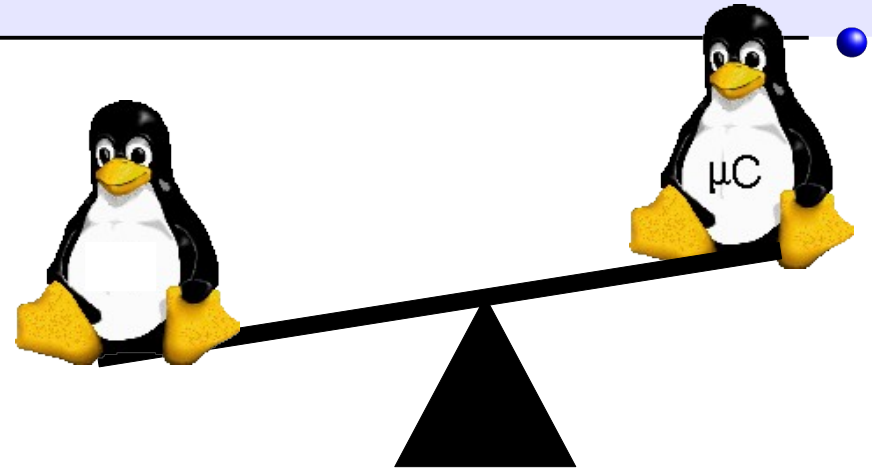
Reasons for using uClinux (2)

- ▶ User access to the hardware
User applications can access the whole system, including device registers.
- ▶ Full **Linux** API
Can use most **Linux** system calls with minor exceptions. Ported applications distributed with **uClinux**.
- ▶ Full **Linux** 2.6 kernel features
stability, preemptible kernel, drivers...
- ▶ Full multi-tasking
Just minor limitations
- ▶ Supported on many processors, which wouldn't be supported by **Linux** otherwise.
See <http://www.uclinux.org/ports/>
Even running on DSP processors (ADI Blackfin, TI DM64x)!



uClinux weaknesses

- ▶ Less momentum than **Linux**.
Much smaller community.
- ▶ Much less on-line documentation and resources available
- ▶ Lack of updates on pages and deliverables on <http://uclinux.org>. Lots of links and resources older than 2002.
- ▶ Lots of projects still using Linux 2.4.



However **uClinux** development is still active: kernel and distribution. **uClinux** releases available for each **Linux** 2.6.x version, released just a few weeks after.



uClibc

<http://www.uclibc.org/> for CodePoet Consulting

- ▶ Lightweight C library for small embedded systems, with most features though.
- ▶ Originally developed for [uClinux](#). Now an independent project.
- ▶ The whole [Debian Woody](#) (thousands of programs) was recently ported to it... You can assume it satisfied most needs!
- ▶ Example size (ARM): approx. 400K (vs. 1700 K for [glibc](#))
- ▶ [uClibc](#) vs. [glibc](#) size comparison (busybox example, static build): 311 K vs. 843 K!



uClinux limitations

In a nutshell

- ▶ Virtual memory = physical memory
- ▶ Fixed memory for processes, can't fragment the memory: more memory consumption
- ▶ No memory protection. No distinction between user mode and kernel mode. Any program can corrupt kernel memory and even cause a system crash.

Very useful details (by David McCullough):

<http://www.linuxjournal.com/article/7221>



Introduction to uClinux

uClinux implementation details



No memory management

- ▶ No virtual memory
Programs addresses need to be preprocessed (“relocated”) before running to obtain unique address spaces.
- ▶ No on-demand paging
Need to load whole program code in RAM
(instead of just loading pages when they are effectively accessed).
- ▶ No memory protection
Any program can crash another program or the kernel. Corruption can go unnoticed and surface later... difficult to track down!
Design your code carefully. Be careful of data from the outside!
- ▶ No swapping
Not really an issue on the tiny embedded devices



Better performance

uClinux can be significantly faster than Linux on the same processor!

- ▶ MMU operation can represent a significant time overhead. Even when an MMU is available, it is often turned off in systems with real-time constraints.
- ▶ Context switching can be much faster on uClinux. On ARM9, for example, the VM based cache has to be flushed at each context switch. No such need when all the processes share the same address space. See an interesting benchmark from H.S. Choi and H.C. Yun: http://opensource.samsung.com/document/uc-linux-04_sait.pdf



Different executable format

- ▶ Standard formats (such as [ELF](#)) rely on VM to create the address space of a process.
- ▶ [flat](#) format: condensed executable format storing only executable code and data, plus the relocations needed to load the executable into any location in memory.
- ▶ [uClinux](#) specific toolchains are needed to create executables in this format.



Different mmap implementation

- ▶ Unless the file is stored sequentially and contiguously, **mmap** needs to allocate memory! Only **romfs** can guarantee this.
- ▶ Another condition is that the storage can directly be accessed in the CPU physical address space. Can work with flash or ROM, but not with disk storage.
- ▶ Only read-only mappings can be shared (no copy-on-write) without allocating memory.
- ▶ In a nutshell, the **mmap** system call is available, but application developers should be aware that it has performance issues in the cases mentioned above.



Other kernel differences

▶ No `tmpfs`

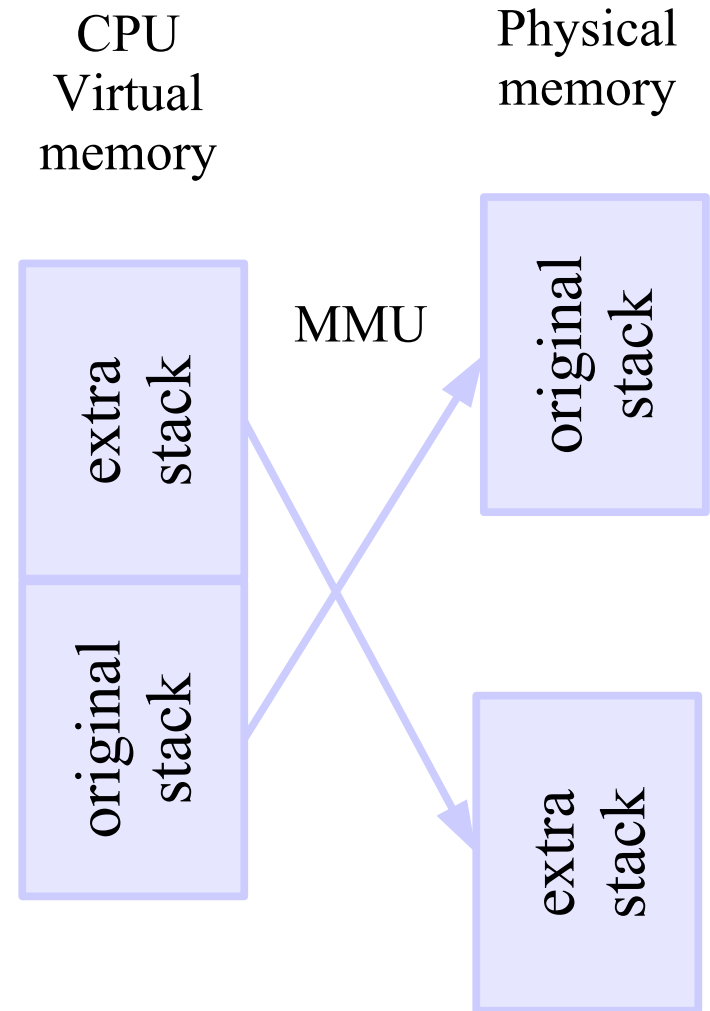
Cannot use the `tmpfs` filesystem relying on VM.
Need to use fixed size ramdisks.



No dynamic stack (1)

Linux

- ▶ With VM, can grow the stack of a running process whenever needed.
- ▶ Whenever an application tries to write beyond the top of its stack, the MMU raises an exception. This causes some new memory to be allocated and mapped in at the top of the stack.



No dynamic stack (2)

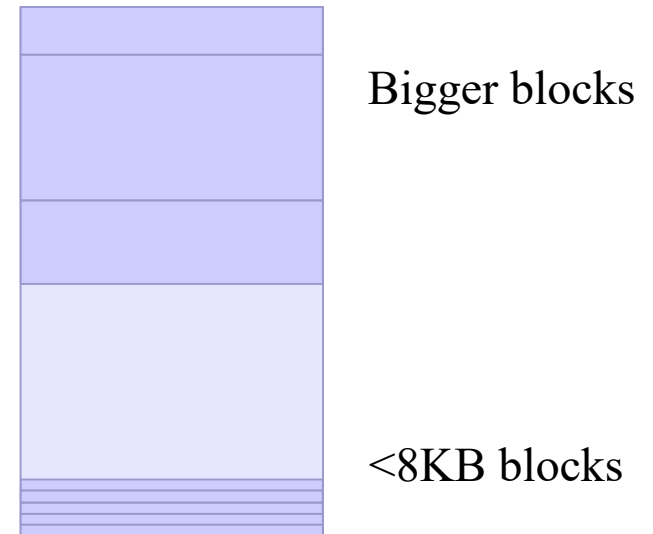
uClinux

- ▶ Stack size must be allocated at compile time: 4 KB by default.
- ▶ No exception raised when a process writes beyond the top of its stack!
The consequences of this could surface much later.
- ▶ If strange crashes happen, try to increase the stack size of programs:
 - ▶ Either recompile:
run `export FLTFLAGS=-s <stacksize>` before recompiling.
 - ▶ Or run `flthdr -s <stacksize> <executable>`



Memory allocation

- ▶ Standard **Linux** allocator: allocates blocks of 2^n size. If 65 KB are requested, 128KB will be reserved, and the remaining 63KB won't be reusable in **Linux**.
- ▶ uClinux 2.4 memory allocator: **kmalloc2** (aka **page_alloc2**)
 - ▶ Allocates blocks of 2^n size until 4KB
 - ▶ Uses 4KB pages for greater requests
 - ▶ Stores amounts not greater than 8KB on the start of the memory, and larger ones at the end. Reduces fragmentation.
 - ▶ Not available yet for Linux 2.6!



No dynamic process size (1)

Linux

- ▶ With VM, can increase or decrease the process size with the `brk()` and `sbrk()` system calls.
- ▶ `malloc()` implementation based on `brk()` and `sbrk()`.



No dynamic process size (1)

uClinux

- ▶ Memory has to be allocated from a global, shared memory pool
- ▶ Different `malloc()` implementation (`malloc-simple()`), accessing memory in this pool, managed by the kernel allocator.
- ▶ Fragmentation: can be unable to allocate enough contiguous memory
Such situations can be detected through `/proc/mem_map` (`kmalloc2`)
- ▶ See <http://www.cyberguard.info/snapgear/tb20020530.html> for details about allocating memory in uClinux.



Can't allocate memory for this extra block,
while it's less than half the free memory!



Tips for reducing memory fragmentation

- ▶ Have your programs allocate smaller chunks of memory, rather than allocating big ones at once.
- ▶ If possible, stop and restart applications when memory is too fragmented. Design your applications so that they can be shut down and restarted.



Tips for avoiding memory issues

Memory issues can be very difficult to track in **uClinux**.

- ▶ Fortunately, they should only happen with your own applications, not with widely tested tools from your **uClinux** distribution.
- ▶ Design with care and with a low memory budget in mind.
- ▶ Idea: first develop and profile your application on Linux (typically on your **i386** desktop). There are several utilities to detect memory issues: **Valgrind**, **memcheck**, **ElectricFence**. See <http://free-electrons.com/articles/swdev> for details.



No fork()

▶ Linux fork()

The child process is a clone of the parent, using the same virtual memory space. New memory allocation happens for the child only when it modifies a page (“copy on write”).

▶ uClinux only implements vfork()

- ▶ The parent execution is stopped and new memory is created before the child process is executed. Consumes more memory!
- ▶ Need to replace all `fork()` calls by `vfork()`
- ▶ No significant impact on multitasking though.



Execute In Place (XIP)

- ▶ Allows to start an application without loading it in RAM.
- ▶ Applies also to multiple instances of the same program.
Saves a lot of RAM!
- ▶ Only supported by `romfs`
(need continuous, non compressed storage).
- ▶ Only supported by the Position-Independent Code (PIC) flavor of the `flat` format (must be supported by the compiler).
- ▶ Caution: XIP may be much slower if storage access time is high.



Shared libraries

- ▶ Pretty different under **uClinux**.
- ▶ Different compiling options...
Making your own won't be familiar.
- ▶ Must be compiled for XIP. Without XIP, shared libraries result in a full copy of the library for each application using it, which is worse than statically linking your applications.

See http://elinux.org/UClinux_Shared_Library for implementation details.



uClinux and Linux 2.6

- ▶ Most of **uClinux** code now merged with mainstream **Linux**.
- ▶ Architectures supports in mainstream Linux:
MMU-less m68k and arm, Hitachi's H8/300 series, NEC v850 processor, ADI Blackfin
- ▶ **Linux** 2.6 can be built with no virtual memory system in a few platforms (not supported on x86, for example).



Introduction to uClinux

Using uClinux



Free Electrons

Introduction to uClinux
© Copyright 2004-2008, Free Electrons
Creative Commons Attribution-ShareAlike 2.5 license
<http://free-electrons.com>

Jul 25, 2025



uClinux on m68knommu platforms

<http://uclinux.org/ports/coldfire/>

- ▶ Kernel sources with the standard **Linux** kernel ([arch/m68knommu/](#))
- ▶ Supported processors: pretty long list! (see [arch/m68knommu/Kconfig](#))
- ▶ Binary filesystem images also available on <http://uclinux.org/ports/coldfire/binary.html>



uClinux on ARM MMU-less platforms

Now supported in mainstream Linux

- ▶ Used to be available as separate patches until version 2.6.20 (approximately)
- ▶ No separate **armnommu** architecture
- ▶ Supported processors:
Look for **!MMU** in **arch/arm/mm/Kconfig**



uClinux on ADI Blackfin

<http://blackfin.uclinux.org/>



- ▶ Supported in mainstream Linux since 2.6.22
- ▶ A nice site and an active developer community
- ▶ Ships binary bootloader, kernel and distribution images.
- ▶ People from other **uClinux** ports can learn from their resources and experience.
- ▶ Actually, **Blackfin** has no VM, but some memory protection. See a nice article on **Blackfin** specifics:

http://docs.blackfin.uclinux.org/doku.php?id=operating_systems#introduction_to_uclinux



Free Electrons

Introduction to uClinux
© Copyright 2004-2008, Free Electrons
Creative Commons Attribution-ShareAlike 2.5 license
<http://free-electrons.com>

Jul 25, 2025



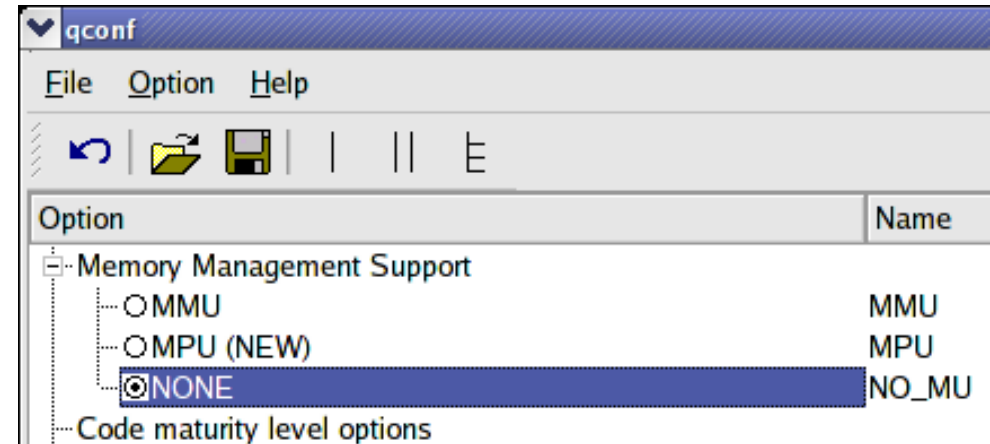
uClinux on other architectures

- ▶ uClinux on the MicroBlaze FPGA processor
<http://www.itee.uq.edu.au/~jwilliams/mblaze-uclinux/>
Nice and active community. Commercial support also available.
- ▶ More resources available on <http://uclinux.org/ports/>
(Caution: lots of broken hyperlinks!)



Disabling the MMU on CPUs with MMU

- ▶ Kernel configuration option
- ▶ Allows to disable the MMU on **some** processors with an MMU (example: Samsung ones)
- ▶ Not supported on **x86**
- ▶ Check by yourself on your own platform!



The uClinux-dist distribution

- ▶ Not a binary distribution, but a tool and sources to build a root filesystem.
- ▶ Contains application sources, patched for compliance with uClinux. 270 applications in **user/** (including BusyBox) and 48 libraries in **lib/**, plus the **uClibc** source code.
- ▶ Also includes kernel sources for a supported 2.2, 2.4 and 2.6 kernel versions (**2.6.25-uc0** in **uClinux-dist-20080808**). This includes code for architectures not supported in the mainline kernel (example: **nios2nommu**).
- ▶ Example size (**uClinux-dist-20080808**):
Compressed archive (**.tar.bz2**): 281 MB!
Expanded archive: 1.5 GB!



uClinux-dist downloads

uClinux.org distribution

- ▶ <http://uclinux.org/pub/uClinux/dist/>
- ▶ For faster download time, can also use
<http://sourceforge.net/projects/uclinux/download>

Best places to find application sources for use with uClinux, even if you don't use the distributions.



uClinux toolchains

Need to get uClinux specific toolchains

- ▶ Toolchain locations are given on <http://uclinux.org/pub/uClinux/dist/>
- ▶ Regular toolchain (compiled to support PIC and XIP), plus flat format utilities: `elf2flt` and `flthdr`.



Toolchain installation example

Arm example

- ▶ Download the arm toolchain from <http://ftp.snapgear.org/pub/snapgear/tools/arm-linux/>
- ▶ Extract the archive
`cd /`
`sudo tar xzf arm-linux-tools-20070808.tar.gz`
Caution: this one installs binaries directly in `/usr/local/bin`!
- ▶ Caution: old `gcc 3.x` toolchains are not relocatable!
They cannot be moved to another location
(unless you create links in the default location)

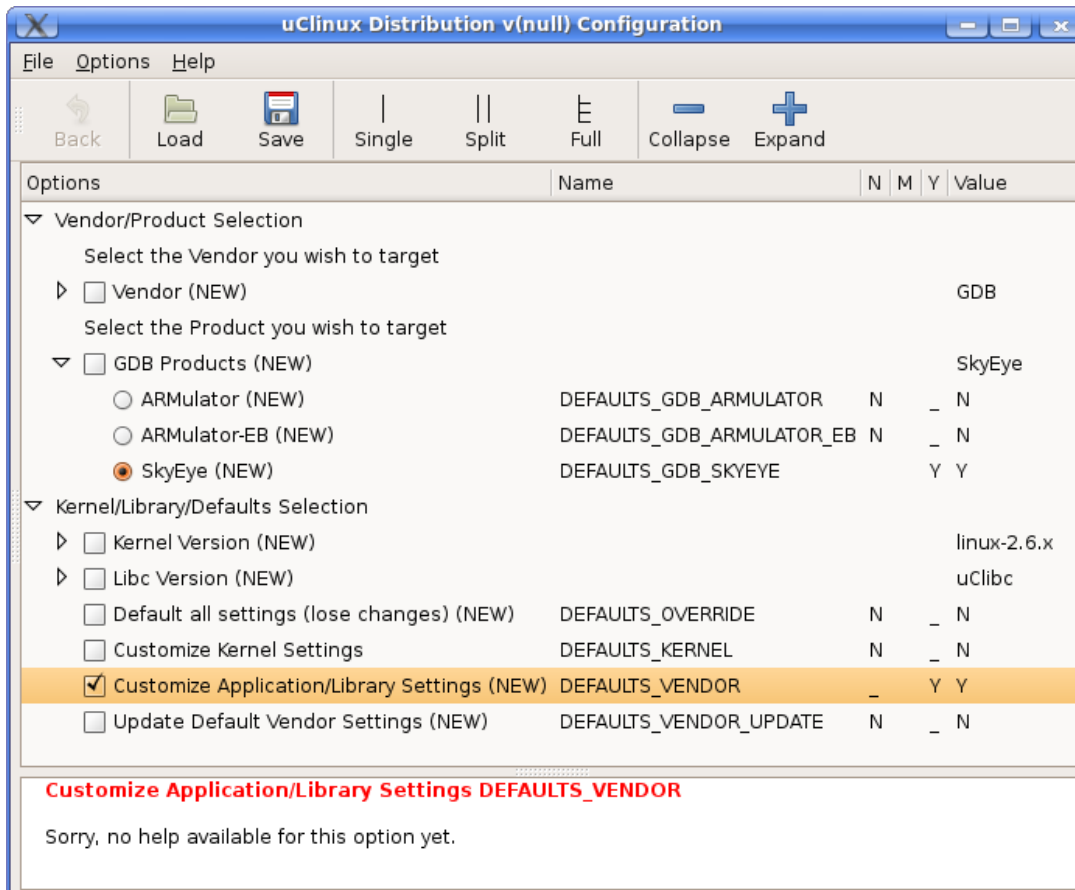


Compiling the uClinux distribution (1)

- ▶ Add your uClinux cross-compiling toolchain to your PATH:
`export PATH=path/bin:$PATH`
(not needed with our toolchain in `/usr/local`)
- ▶ Install the `genromfs` tool if you don't have it:
`apt-get install genromfs` (Debian / Ubuntu)
- ▶ In the toplevel `uClinux-dist` directory:
`make xconfig`
or `make menuconfig`



Compiling the uClinux distribution (2)



- ▶ Choose your platform (vendor and product).
- ▶ Choose your kernel and C library version.
- ▶ Customize or not kernel and tool settings.



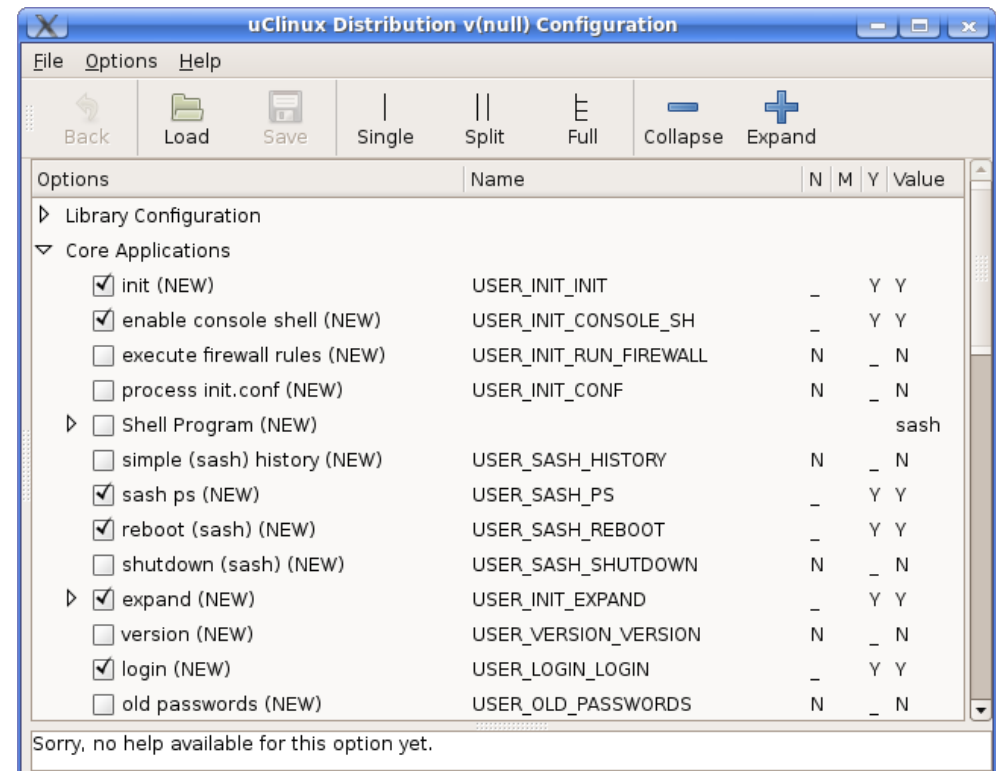
Compiling the uClinux distribution (3)

Other interfaces pops up

- ▶ Configure your kernel (if you chose this option)
- ▶ Configure the user tools (if you chose this option)

▶ Compile:
make

This generates binary images for your target in the **images/** directory.



Adding your custom application (1)

The easiest way is to add your application to **uClinux-dist**

Hello world example:

▶ Create a **user/hello** directory in **uClinux-dist**

▶ Create a **user/hello/hello.c** file:

```
#include <stdio.h>
int main() {
    printf("Hello, world\n");
    return 0;
}
```



Adding your own application (2)

- ▶ Create `user/hello/Makefile`:
`EXEC = hello`

```
# Stack size (bytes)  
FLTFLAGS += -s 64
```

```
all: $(EXEC)
```

```
romfs:  
    $(ROMFSINST) /bin/$(EXEC)
```

```
clean:  
    -rm -f $(EXEC) *.elf *.gdb *.o
```

- ▶ See other Makefile examples in the `user/` directory.



Adding your custom application (3)

- ▶ Declare your application somewhere in `user/Kconfig`:
(for example in `Miscellaneous Configuration`)
`config USER_HELLO`
 `bool "Hello world program"`
 `help`
 A sample C program
- ▶ Add the below line to `user/hello/Makefile`:
`dir_$(CONFIG_USER_HELLO) += hello`
- ▶ Run `make menuconfig` or `make xconfig` and enable your new application.
- ▶ Run `make` to compile your application and add it to your filesystem image.



Summary

- ▶ Worthy to use **uClinux** (rather than a proprietary RTOS) on processors without an MMU. Surprisingly, with **uClinux**, there are only minor differences with what you can do with a full Linux kernel.
- ▶ In some cases, worthy to use **uClinux** rather than **Linux** on processors with an MMU, for performance reasons.
- ▶ Though a lot of work has already been done and most applications have already been ported, some **uClinux** experience or learning is definitely needed when you start a new project.



Resources

- ▶ uClinux, State Of The Nation, by Greg Ungerer (Apr. 2007)
<http://www.linuxdevices.com/files/article078/uclinux-sotn.pdf>
- ▶ uClinux development mailing list
<https://mailman.uclinux.org/mailman/listinfo/uclinux-dev>
Archives: <http://mailman.uclinux.org/pipermail/uclinux-dev/>
- ▶ <http://ucdot.org/>
A wealth of resources, FAQs, forums for uClinux developers!
Don't miss the very complete FAQ:
<http://www.ucdot.org/faq.pl>



Free alternatives to uClinux

eCos: <http://en.wikipedia.org/wiki/ECos>



- ▶ Lightweight real-time embedded system, originally contributed by Red Hat / Cygnus solutions.

FreeRTOS: <http://en.wikipedia.org/wiki/FreeRTOS>

- ▶ Another free RTOS, gaining popularity over eCos in the last years. Apparently, eCosCentric seems to put more focus on their proprietary extensions.



Both also support 16 bit processors
(not supported by uClinux and Linux)

